



EAST WEST UNIVERSITY

Project Report

“Algorithm Implementation for Travel Route Optimization”

Semester: Fall-2024

Course Title: Algorithms

Course Code: CSE246

Submitted by

Shawna Akter

ID: 2023-1-60-071

Sec: 07

Tabassum Nahar Yeah

ID: 2023-1-60-068

Sec: 10

Submitted to

Amit Mandal

Lecturer

Department of Computer Science and Engineering

Date: 02 February, 2025

Abstract

This project involves the implementation of an algorithm for selecting optimal travel routes based on a given budget. The system generates random routes between cities, and two optimization techniques—Knapsack and Coin Change—are employed to select the best possible routes within a budget. Merge Sort is also utilized to sort the routes by distance. The program measures the execution time of these operations, providing insights into performance.

Introduction

In this project, we simulate a scenario where a traveler wants to select the most optimal routes between a set of cities. The traveler has a limited budget and aims to maximize the total distance traveled, given the constraints. The goal of the project is to demonstrate the application of two dynamic programming algorithms (Knapsack and Coin Change) to solve the route selection problem efficiently.

Problem Statement

Given a set of cities with various travel routes connecting them, each route has an associated distance and cost. The task is to find the set of routes that maximizes the total distance traveled without exceeding the given budget. Furthermore, we sort the routes based on distance to ensure that we always prioritize shorter routes first, and analyze the computational efficiency of different parts of the algorithm.

Methodology

Data Generation

The program generates random travel routes between a fixed set of cities. For each route, it assigns a random distance (between 1 and 1000) and a random cost (between 1 and 1000). The cities are predefined and stored in an array. The program ensures that a route's source and destination are distinct by selecting two different cities at random.

Algorithms

- 1. Merge Sort** - Sorts the routes based on distance.
- 2. 0/1 Knapsack Problem (Dynamic Programming)** - Selects routes to maximize distance within a given budget.
- 3. Coin Change Problem (Dynamic Programming)** - Finds the minimum number of routes that sum up to the budget.

(a) Merge Sort - Sorting Routes by Distance

Purpose: The program sorts the travel routes based on their distance in ascending order.

Algorithm Used: Merge Sort (a divide-and-conquer sorting algorithm).

Function:

```

void merge_sort(Route arr[], int l, int r) {

    if (l < r) {
        int m = l + (r - l) / 2;
        merge_sort(arr, l, m);
        merge_sort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

```

Steps:

1. The array is recursively divided into two halves until each sub-array has one element.
2. The sub-arrays are then merged back in sorted order.
3. This process continues until the entire array is sorted.

Time Complexity:

Best Case: $O(n \log n)$

Average Case: $O(n \log n)$

Worst Case: $O(n \log n)$

Reason: The merge step takes $O(n)$ time, and since we divide the array $\log n$ times, the total complexity is $O(n \log n)$.

(b) 0/1 Knapsack Algorithm - Maximizing Total Distance within Budget

Purpose: The program selects a subset of routes such that their total cost does not exceed the budget while maximizing the total distance.

Algorithm Used: 0/1 Knapsack Dynamic Programming (DP)

Function:

```
int knapsack(Route routes[], int n, int budget, int selected[]) {  
    int **dp = malloc((n + 1) * sizeof(int *));  
    for (int i = 0; i <= n; i++) {  
        dp[i] = malloc((budget + 1) * sizeof(int));  
    }  
  
    for (int i = 0; i <= n; i++) {  
        for (int j = 0; j <= budget; j++) {  
            if (i == 0 || j == 0)  
                dp[i][j] = 0;  
            else if (routes[i - 1].cost <= j)  
                dp[i][j] = (dp[i - 1][j] > dp[i - 1][j - routes[i - 1].cost] + routes[i - 1].distance) ? dp[i - 1][j] : dp[i - 1][j - routes[i - 1].cost] + routes[i - 1].distance;  
            else  
                dp[i][j] = dp[i - 1][j];  
        }  
    }  
}
```

Steps:

1. Create a 2D DP table $dp[i][j]$, where i represents the number of routes considered, and j represents the budget.
2. If the cost of a route is within the budget, decide whether to include it or not:
Include it: $dp[i][j] = dp[i-1][j - \text{cost}] + \text{distance}$
Exclude it: $dp[i][j] = dp[i-1][j]$
3. After filling the table, backtrack to find which routes were selected.

Time Complexity:

$O(n \times \text{budget})$

Reason: The DP table has n rows (number of routes) and budget columns. Each cell takes constant $O(1)$ time.

(c) Coin Change Algorithm - Minimizing the Number of Routes within Budget

Purpose: The program selects routes so that the total cost stays within the budget while minimizing the number of routes used.

Algorithm Used: Dynamic Programming for Coin Change

Function:

```
int coin_change(Route routes[], int n, int budget, int selected[]) {
    int *dp = malloc((budget + 1) * sizeof(int));
    for (int i = 0; i <= budget; i++) {
        dp[i] = INT_MAX; // Initialize with a large value
    }
    dp[0] = 0;

    for (int i = 1; i <= budget; i++) {
        for (int j = 0; j < n; j++) {
            if (routes[j].cost <= i && dp[i - routes[j].cost] != INT_MAX) {
                if (dp[i - routes[j].cost] + 1 < dp[i]) {
                    dp[i] = dp[i - routes[j].cost] + 1;
                }
            }
        }
    }
}
```

Steps:

1. Create a DP array `dp[budget+1]` initialized to a large value (`INT_MAX`)
2. Set `dp[0] = 0` (zero cost requires zero routes).
3. Iterate through each budget amount and update `dp[i]` to store the minimum number of routes needed.
4. Backtrack to find the selected routes.

Time Complexity:

$O(n \times \text{budget})$

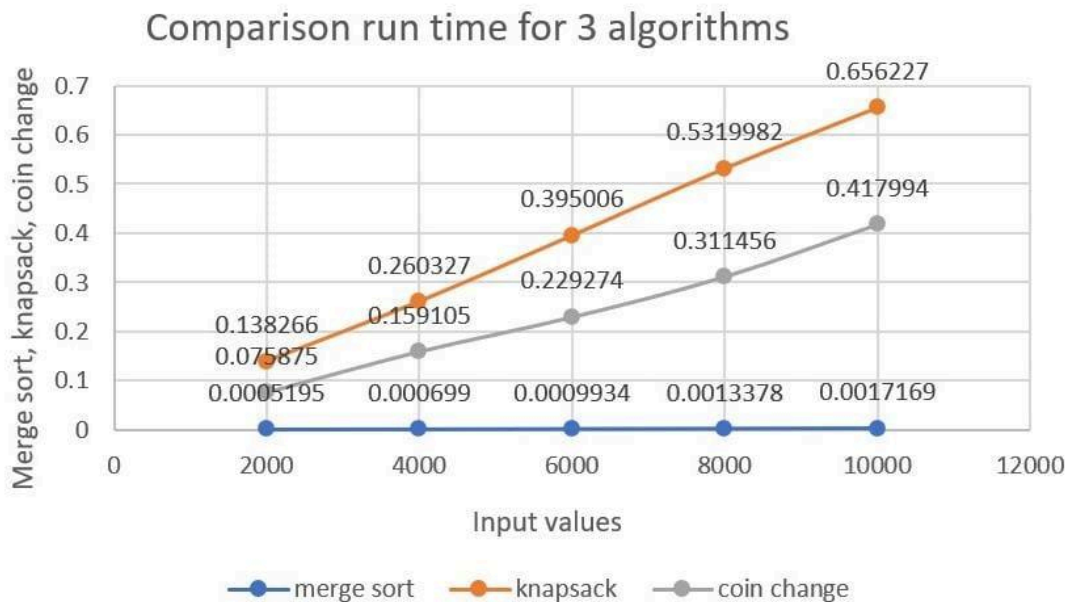
Reason: Similar to Knapsack, we iterate through n routes and process each budget amount, leading to $O(n \times \text{budget})$ complexity.

Overall Time Complexity Summary

If the budget is large, $O(n \times \text{budget})$ can become slow. The worst-case scenario occurs when both Knapsack and Coin Change take too much time due to large budget values.

Algorithms	Time Complexity
Merge Sort	$O(n \log n)$
0-1 Knapsack DP	$O(n * \text{budget})$
Coin Change DP	$O(n * \text{budget})$

Graph



Conclusion

- Merge Sort efficiently sorts the routes.
- Knapsack selects routes to maximize distance within budget.
- Coin Change minimizes the number of routes needed for exact budget usage.

The program compares the efficiency of these approaches and stores results in files.

